hashlock.

Security Audit

Ponz (MemeCoin Launchpad)

June 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Ponz team partnered with Hashlock to conduct a security audit of their smart contract. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

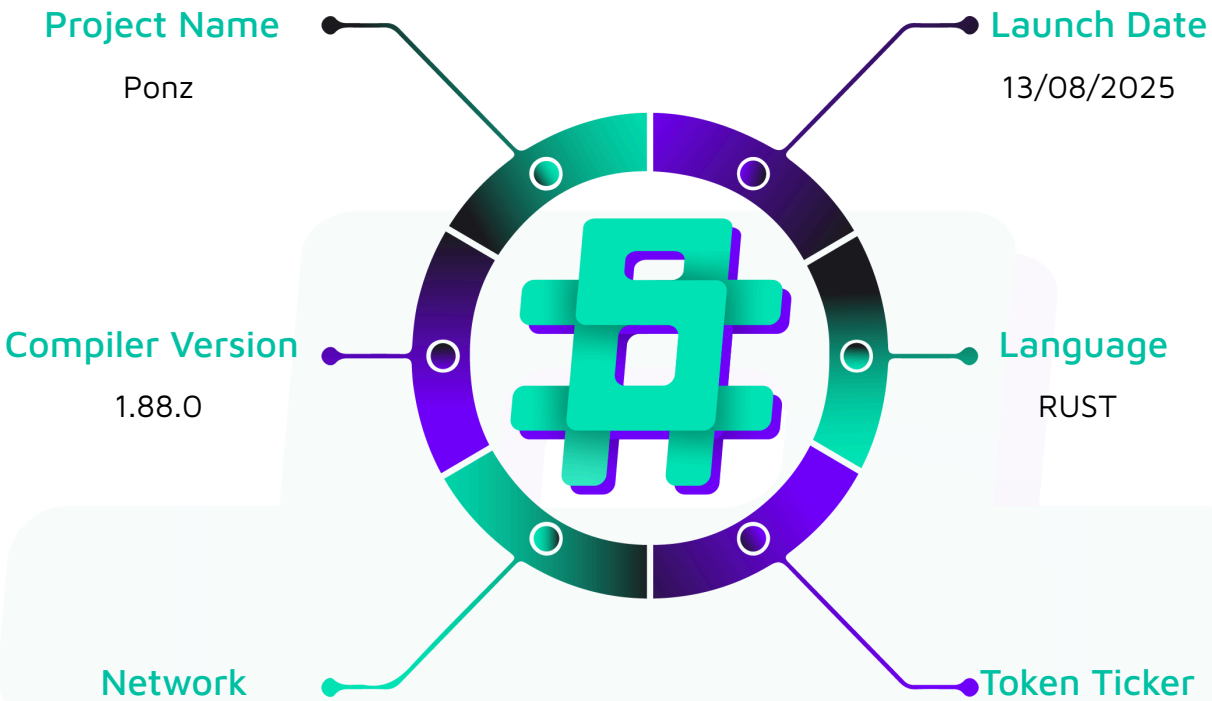
Project Context

Ponz is a memecoin launchpad that enables users to create coins instantly and for free with fully custom tokenomics in a single click. Unlike traditional launchpads, coins launched on Ponz can be configured with automatic SOL rewards every 5 minutes, jackpot, burn, and smart dev lock, all of which can be toggled and adjusted by the creator.

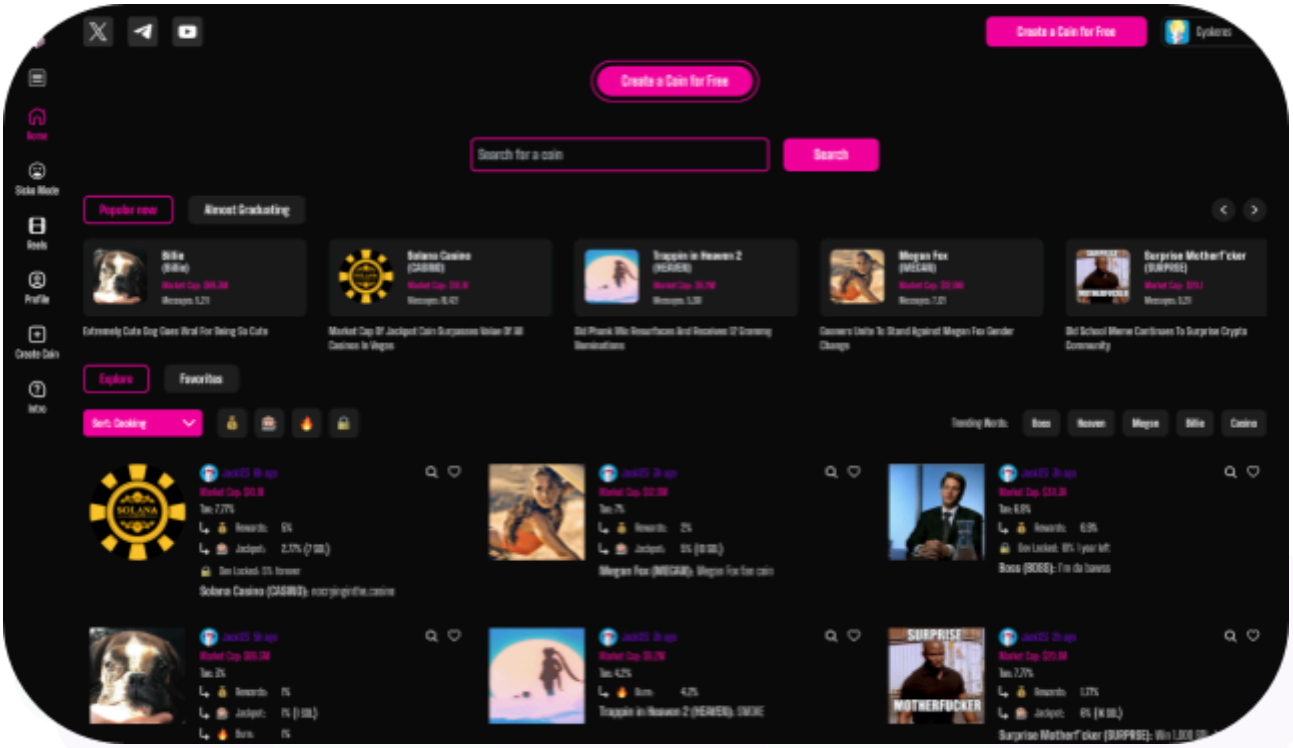
Ponz is also the first launchpad to integrate reels, offering coins continued discovery after launch.

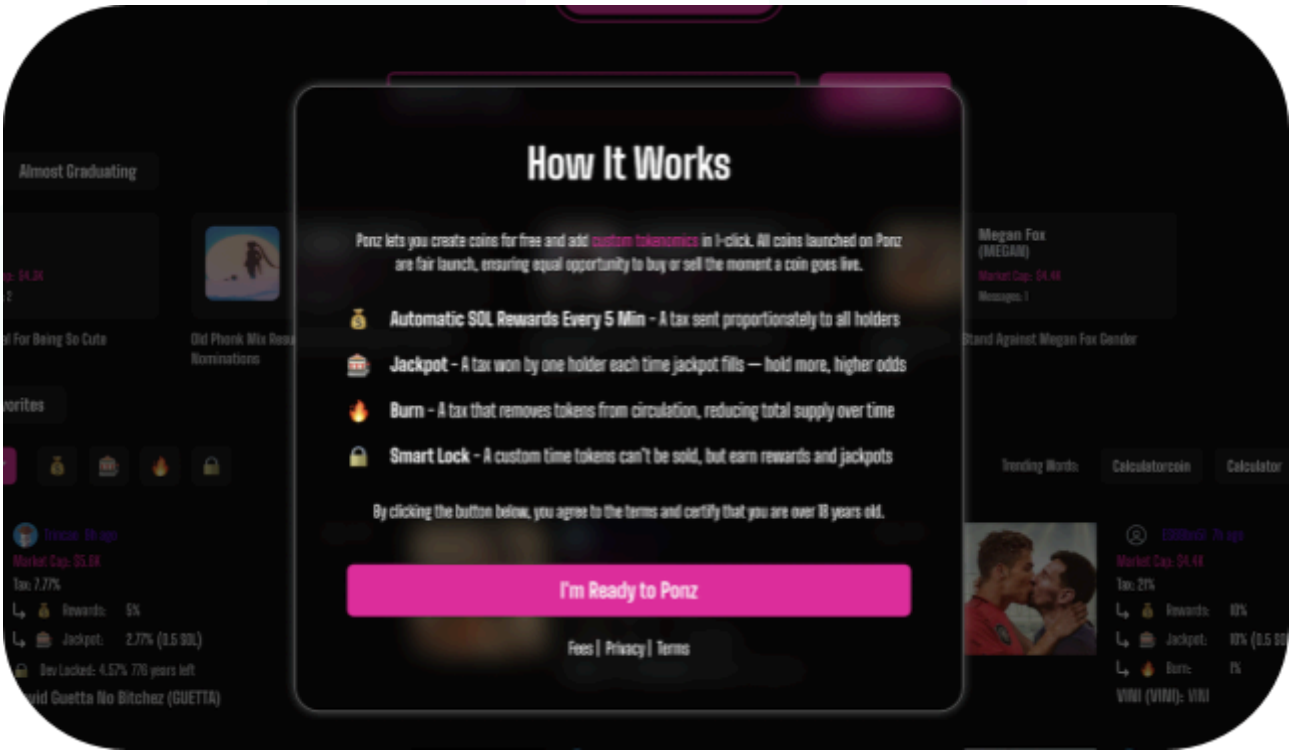
Project Name: Ponz
Project Type: Memecoin Launchpad
Website: https://ponz.app/
Branding:

Visualised Context:



Project Visuals:





Audit scope

We at Hashlock audited the rust code within the Ponz project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Ponz Smart Contracts
Platform	Solana / Rust
Audit Date	June, 2025
Contract 1	accept_update_wallet.rs
Contract 2	create_token_pool.rs
Contract 3	init_master.rs
Contract 4	init_sol_pool.rs
Contract 5	update_wallet.rs
Contract 6	withdraw_sol.rs
Contract 7	master.rs
Contract 8	sol_pool.rs
Contract 9	buy_lock.rs
Contract 10	buy.rs
Contract 11	create_token.rs
Contract 12	init_configuration.rs
Contract 13	init_fee_pool.rs
Contract 14	remove_liquidity.rs
Contract 15	sell.rs
Contract 16	update_configuration.rs
Contract 17	withdraw_fee_pool.rs

Contract 18	withdraw_token_pool_lock.rs
Contract 19	bonding_curve.rs
Contract 20	fee_pool.rs
Contract 21	init_configuration.rs
Audited GitHub Commit Hash	a785902eb2974d9d878dd5bf209c7ea88a9f8c19
Fix Review GitHub Commit Hash	e29f95a2c4b40d5c4a56e40bc0f4c35d01b3eff1

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The general security overview is presented in the Standardised Checks section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

7 High severity vulnerabilities

3 Medium severity vulnerabilities

2 Low severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
accept_update_wallet.rs - Allows a pending owner/system/mint-authority/multisig wallet to accept and finalize an update.	**Contract achieves this functionality.**
create_token_pool.rs - Creates the vault's token pool PDA under a given mint.	**Contract achieves this functionality.**
Init_master.rs - Initializes the Ponz-Vault master account (setting multisig wallet).	**Contract achieves this functionality.**
Init_sol_pool.rs - Initializes the Ponz-Vault SolPool to collect native-SOL fees.	**Contract achieves this functionality.**
update_wallet.rs - Schedules a pending update to one of the four global wallets (owner, system, mint-auth, multisig).	**Contract achieves this functionality.**
withdraw_sol.rs - Allows the multisig to withdraw all available SOL from the SolPool.	**Contract achieves this functionality.**
master.rs - Defines the on-chain Master account's state and seed.	**Contract achieves this functionality.**
sol_pool.rs - Defines the on-chain SolPool account's state and seed.	**Contract achieves this functionality.**

buy_lock.rs - Only the token-creator may buy & lock a batch of tokens for a fixed time.	**Contract achieves this functionality.**
buy.rs - Allows any user to exchange SOL → tokens at the curve price, with slippage protection.	**Contract achieves this functionality.**
Create_token.rs - Creates a new bonding curve: mint, token pools, metadata, mints, then burns the mint-authority.	**Contract achieves this functionality.**
Init_configuration.rs - Initializes the global configuration (fees, limits, wallets).	**Contract achieves this functionality.**
Init_fee_pool.rs - Initializes the FeePool PDA to collect swap fees.	**Contract achieves this functionality.**
Remove_liquidity.rs - Lets the system wallet withdraw all SOL from a completed bonding curve, returning tokens to the user.	**Contract achieves this functionality.**
sell.rs - Allows users to swap tokens → SOL at the curve price and pay a swap fee.	**Contract achieves this functionality.**
update_configuration.rs - Allows the owner or system wallet to adjust swap fees & curve parameters.	**Contract achieves this functionality.**
withdraw_fee_pool.rs - Allows the multisig wallet to withdraw accumulated SOL fees from FeePool.	**Contract achieves this functionality.**

withdraw_token_pool_lock.rs - Let the system wallet reclaim any expired locked tokens.	**Contract achieves this functionality.**
bonding_curve.rs - Implements the core curve math (init, price, token_out, sol_out, state).	**Contract achieves this functionality.**
fee_pool.rs - Defines the on-chain FeePool account's schema and seed.	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Ponz project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Ponz project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] buy_lock.rs - Unrestricted buy_lock access

Description

The `buy_lock` instruction intends that it should allow only the creator of the token to buy an amount of tokens that will be locked.

In practice, anyone with `SOL` can call `buy_lock`, because there is no on-chain check binding `ctx.accounts.payer` to the `BondingCurve.creator` field.

Vulnerability Details

What the code does:

```
// In BuyLockCtx, `payer: Signer<'info>` has no further constraint
pub payer: Signer<'info>,
// ...
pub fn process(ctx: Context<BuyLockCtx>, ...) -> Result<()> {
    // No check here to ensure `payer` == `bonding_curve.creator`
    // ALL users can invoke this path.
    // ...
    ctx.accounts.bonding_curve.unlock_time = Clock::get()?.unix_timestamp + lock_time;
}
```

The program never verifies that the signer of `buy_lock` is the on-chain `creator` of the token. As a result, any user can call this instruction and manipulate the shared lock state.

Impact

Any user (malicious or otherwise) can invoke `buy_lock`, locking tokens and setting an arbitrary `unlock_time`. They could:

- Extend the lock window for legitimate lockers
- Confuse the downstream logic that assumes only the creator ever locks
- Force off-chain discrepancies

Recommendation

Enforce strict creator-only access on `buy_lock` by adding a simple authority check at the top of the instruction:

```
// In buy_lock::process(...)

if ctx.accounts.payer.key() != ctx.accounts.bonding_curve.creator {

    return Err(PonzError::UnAuthorized.into());

}
```

Status

Resolved

[H-02] Mutable lock duration in buy_lock allows repeated overwrites

Description

The `buy_lock` instruction sets a global `unlock_time` for token locks each time it's called, but never checks if a lock is already active. As a result, the lock duration can be repeatedly altered—shortened or extended—after the initial lock, thereby violating the intended behavior.

Vulnerability Details

What the code does:

```
// In buy_lock::process(...)

let timestamp = Clock::get()?.unix_timestamp;

ctx.accounts.bonding_curve.unlock_time = timestamp

    .checked_add(lock_time)

    .ok_or(PonzError::MathOverflow)?;
```

Each invocation unconditionally writes `unlock_time = now + lock_time`, regardless of any existing future timestamp.

There is no guard to prevent a second (or third…) `buy_lock` call from updating `unlock_time`. The code never checks whether `unlock_time` has already been set to a later moment.

Impact

Even when restricted to the creator role, the creator can maliciously or accidentally change their own lock after it begins, either shortening it to retrieve tokens prematurely or extending it indefinitely.

This breaks the guarantee that "once set, the lock time is final," eroding user trust and potentially causing business logic errors in downstream systems that rely on stable lock periods.

Recommendation

Before writing the new deadline, reject any `buy_lock` if an existing lock remains unexpired. For example, at the top of the process(...):

```
let now = Clock::get()?.unix_timestamp;

// Disallow overwriting an active lock

if ctx.accounts.bonding_curve.unlock_time > now {

    return Err(PonzError::InvalidLockTime.into());

}
```

Status

Resolved

[H-03] DoS via pre-funded Associated Token Account in buy_lock

Description

The `buy_lock` instruction in the Ponz program attempts to lazily initialize the buyer's Associated Token Account (ATA) by checking whether the ATA `PDA` has a zero `SOL` balance.

An attacker can pre-fund that `PDA` with a trivial amount of `SOL`, causing the initialization step to be skipped and the subsequent token transfer to an uninitialized account to revert, fully denying service of the lock-purchase flow to the targeted buyer.

Vulnerability Details

Faulty initialization guard:

```
// In buy_lock::process

if ctx.accounts.payer_ata.lamports() == 0 {

    // Only creates ATA if lamports == 0

    create(/* Associated Token Account CPI */)?;

}

// Then always attempts to transfer tokens

transfer_checked(/* ... into payer_ata */)?;
```

Pre-funding attack vector:

1. Derive the ATA PDA for a victim's wallet and mint (deterministic via Associated Token Program).

Attacker sends a single `lamport` to that `PDA` address.

2. Victim calls `buy_lock`; the `lamports()==0` check fails, so no ATA CPI runs.
3. The ensuing `transfer_checked` into the uninitialized account always reverts, aborting the transaction.

Impact

- Complete DoS of `buy_lock` for any targeted user.
- Minimal cost: ~1 lamport per victim.
- Predictable: ATA addresses are public, making large-scale targeting trivial.

Recommendation

4. Replace the manual `lamports` check with Anchor's built-in pattern `init_if_needed`, like the rest of the protocol:

```
#[account(
  init_if_needed,
  payer = payer,
  associated_token::mint = mint,
  associated_token::authority = payer,
)]
pub payer_ata: InterfaceAccount<'info, TokenAccount>;
```

Status

Resolved

[H-04] DoS via Pre-Funded PDA in create_token flow

Description

The `create_token` instruction unconditionally invokes the System Program's `create_account` CPI to allocate the bonding curve's token-pool PDA.

Since PDA addresses are publicly derivable and require no signature to allocate, an attacker can "pre-create" that PDA ahead of legitimate initialization.

When the real `create_token` call runs, its `create_account` CPI fails with `AccountAlreadyExists`, permanently preventing the bonding curve from ever being created for that mint.

Vulnerability Details

Unconditional PDA creation:

```
// In create_token::process

create_account(

  CpiContext::new_with_signer(

    ctx.accounts.system_program.to_account_info(),

    CreateAccount {

      from: ctx.accounts.payer.to_account_info(),

      to:   ctx.accounts.token_pool.to_account_info(),

    },

    &[&[

      &ctx.accounts.mint.key().to_bytes(),

      TOKEN_POOL_SEED,

      &[ctx.bumps.token_pool],

    ]],

  ),

  lamports,

  space as u64,

  &ctx.accounts.token_program.key(),

)?;
```

The code makes no existence check or catch for `AccountAlreadyExists`.

Anyone can compute the same PDA via the mint's public key and `TOKEN_POOL_SEED`.

Impact

- Permanent Denial of Service: Legitimate creators can never initialize the bonding curve for a given mint, halting that market indefinitely.

- Low attacker cost: A single transaction (and minimal rent-exempt SOL) suffices to block any mint's pool creation.

Recommendation

Use idempotent account creation: Replace raw `create_account` CPI with Anchor's `init_if_needed` or SPL's `create_idempotent`, which checks ownership/data-layout rather than `lamports`.

Status

Resolved

[H-05] Fee-on-transfer misaccounting in sell increases reserves by gross amount

Description

Ponz mints every token with the SPL Token-2022 `TransferFee` extension, which deducts a configurable percentage on each transfer (the fee is sent to a withheld-authority account).

However, the sell instruction blindly adds the gross `token_in_amt` to its internal `token_reserves`, rather than the net amount received. This disconnect causes the on-chain reserves to drift above the true token balance over successive sales.

Vulnerability Details

During token creation, the program calls:

```
transfer_fee_initialize(
  …,
  transfer_fee_basis_points, // e.g. 100 = 1%
  maximum_fee
)?;
```

This makes the token with `fee-on-transfer` functionality. Every `transfer_checked` against that mint deducts (`amount × basis_points / 10 000`), so the pool receives less than `token_in_amt`.

Reserves Update is Gross:

```
// In sell::process
transfer_checked(/*…*/, token_in_amt, /*…*/)?;   // user → pool, fee deducted
ctx.accounts.bonding_curve.token_reserves =
  ctx.accounts.bonding_curve.token_reserves
    .checked_add(token_in_amt)?;                    // <-- wrong: should be not received
```

This results in the token reserves drifting higher over time.

If `token_in_amt = 10 000` and `fee = 1%` → pool gets `9 900`, but reserves increase by `10 000`.

After `n` sells, reserves are inflated by `n` × `fee`, skewing all future price calculations.

Impact

Distorted pricing since the price calculation uses `token_reserves`

Confusing users - the contract/UI will show X supply available, while in reality, there will be less

Recommendation

Compute net received: After the `transfer_checked` CPI, read the on-chain pool balance:

```
let actual_received = ctx.accounts.token_pool.amount
  .checked_sub(previous_pool_amount)
  .ok_or(PonzError::MathOverflow)?;
```

1. Update reserves with net amount:

```
ctx.accounts.bonding_curve.token_reserves =
  ctx.accounts.bonding_curve.token_reserves
    .checked_add(actual_received)?;
```

Status

Resolved

[H-06] Use of Non-Deterministic Floating Point for fee calculation

Description

The Bonding Curve uses floating point representation for on-chain logic (price, fee, slippage calculations) that could jeopardise consensus because different validator hardware/compilers can round differently. All floating-point maths must be replaced with fixed-point integers.

Vulnerability Details

The bonding curve implementation uses `IEEE-754` floating-point arithmetic (`f32`/`f64`) for critical financial calculations that could have a critical impact on the protocol, including:

Fee computations (`swap_fee` calculations in buy.rs, sell.rs, buy_lock.rs)

```
 let fee_lamports = lamports_in_amt as f64 * ctx.accounts.global_configuration.swap_fee as f64;

let fee_lamports = fee_lamports as u64;  // ← Truncation varies by platform
```

1. Lock percentage calculations (lock_percent in buy_lock.rs)

```
pub fn process(

    ctx: Context<BuyLockCtx>,

    lamports_in_amt: u64,

    expected_token_amt: u64,

    lock_percent: f64,

    lock_time: i64,

) -> Result<()> {
```

`IEEE-754` operations are implemented in software in Solana's `BPF` runtime. Different CPU architectures (`x86-64` vs `ARM`), compiler optimizations, and `LLVM` versions can produce slightly different rounding results for the same mathematical operation. These differences are amplified when results are stored on-chain and reused in subsequent calculations. Therefore, the program runs, but its continued correct execution relies on

all validators happening to use bit-identical helpers – a property the Solana runtime explicitly does not guarantee.

Impact

An `IEEE-754` double `(f64)` has 53 bits of precision, so it can only exactly represent integers up to 2^53 (≈9 × 10^15). `Converting` a u64 lamport amount (say 1 SOL = 1 × 10^9 lamports) to `f64` and back will be exact in that range, but as soon as you multiply by a fractional fee (e.g. 0.003), the result may no longer be representable exactly and will be rounded to the nearest representable `binary64` value. When you cast that back `to u64 as u64`, Rust truncates toward zero rather than rounding to nearest, so you lose whatever fractional lamports the float couldn't capture.

Consider this Calculation for exact values

```
Suppose:

lamports_in_amt = 123_456_789;

fee_lamports   = 123_456_789.0 * 0.0025

               = 308_641.9725  // not an integer!



// After truncation:

fee_lamports_u64 = 308_641  // lost the .9725 fraction
```

The use of Floating Point also increases the cost of CUs exponentially, as they are a super expensive on-chain operation with limitations, as well as risks

References -
https://www.helius.dev/blog/solana-arithmetic
https://solana.com/docs/programs/limitations#float-rust-types-support

Recommendation

Replace all floating-point operations with fixed-point integer arithmetic using a consistent scaling factor (e.g., 1e9 for 9 decimal places). For Example:

```
let fee_lamports = lamports_in_amt

    .checked_mul(25)?      // 123_456_789 × 25 = 3_086_419_725

    .checked_div(10_000)?; // 3_086_419_725 ÷ 10 000 = 308_641
```

Status

Resolved

[H-07] In `buy_lock.rs`, the lock functionality can be bypassed because f64

Description

The buy_lock instruction uses `lock_percent` as an `f64` value, which has the possibility of being a `NaN`. This is particularly concerning as this value will bypass the Valid Lock percent check in the function

Vulnerability Details

The code checks:

```
if lock_percent <= 0.0 ||_percent > 1.0 {

    return Err(PonzError::InvalidLockPercent.into());

}
```

But by IEEE-754 rules:

Any comparison with NaN (`lock_percent <= 0.0`, `lock_percent > 1.0`) returns false.

Therefore, if someone passes NaN, both branches are false, and the check is skipped.

Impact

Multiplication by NaN yields NaN:

```
let lock_token_amount = (estimated_out_token as f64 * lock_percent) as u64;
```

When `lock_percent` is NaN:

- `estimated_out_token as f64 * NaN` ⇒ NaN
- Casting `NaN as u64` in Rust yields `0`.
- User receives all tokens immediately, bypassing the lock mechanism completely

The code then:

- Transfers 0 tokens into the lock account.
- Transfers all purchased tokens to the user's ATA.
- Bypasses the intended lock mechanism entirely

Malicious callers can specify `lock_percent = 0x7ff8_0000_0000_0000` (quiet-NaN) and receive all tokens unlocked, despite the lock logic.

Recommendation

Avoid floating-point for token logic entirely. Use integer math instead, as this design is immune to NaN and floating-point pitfalls, which will also fix H-06

Status

Resolved

Medium

[M-01] Slippage protection bypass on gross amount for fee-on-transfer tokens

Description

The `buy` and `buy_with_lock` instructions compare a user's `expected_token_amt` against the gross `estimated_out_token` before transferring. Because Ponz mints every token with the SPL-Token-2022 `TransferFee` extension, each transfer deducts a fee (e.g. 1%). Buyers who pass the slippage check can end up receiving fewer tokens than their minimum, breaking core user guarantees and opening the door to value loss.

Vulnerability Details

Flawed slippage check:

```
// 1) Calculate gross output

let estimated_out_token = curve.calculate_tokens_out(lamports_in);

// 2) Compare gross to user's minimum

if estimated_out_token < expected_token_amt {

    return Err(PonzError::SlippageExcceded.into());

}

// 3) Transfer gross amount—fee is deducted on-chain

transfer_checked(..., amount = estimated_out_token, ...)?;
```

Intended: Buyer should receive `≥ expected_token_amt`.

Actual: `Buyer` receives `estimated_out_token - fee`, which can be `< expected_token_amt`.

Real world example:

- `expected_token_amt = 10_000`
- `estimated_out_token = 10_050 → passes check`
- `Fee (1%) = 100 → net = 9_950 (< 10_000)`

Impact

- Ineffective slippage protection
- Buyers lose tokens that they explicitly protected themselves against.

Recommendation

Perform a slippage check on the net `estimated-out` amount, accounting for the transfer fee.

Status

Resolved

[M-02] The Curve allows creation of RUG PULL tokens

Description

The `create_token` instruction allows users to create bonding curve tokens with arbitrary `transfer_fee_basis_points` and `maximum_fee` parameters without any validation or upper bounds. This enables the creation of "rug pull" tokens, where every transfer incurs 100% fees that go directly to the token creator.

Vulnerability Details

The instruction accepts user-provided parameters without validation:

```
#[derive(AnchorDeserialize, AnchorSerialize)]
pub struct TokenMetadataArgs {
    pub name: String,
    pub symbol: String,
    pub uri: String,
    pub transfer_fee_basis_points: u16,  // ← No upper bound validation
    pub maximum_fee: u64,                // ← No upper bound validation
}
```

These parameters are directly passed to the SPL Token-2022 `transfer_fee_initialize` function:

```
transfer_fee_initialize(
    CpiContext::new(
        ctx.accounts.token_program.to_account_info(),
        TransferFeeInitialize {
            token_program_id: ctx.accounts.token_program.to_account_info(),
            mint: ctx.accounts.mint.to_account_info(),
        },
```

```
    ),
    None,                                       // update transfer fee authority
    Some(ctx.accounts.withheld_authority.key), // who withdraw transfer fees
    transfer_fee_basis_points,                  // ← Can be set to 65,535 (100%)
    maximum_fee,                                // ← Can be set to u64::MAX
)?;
```

Impact

- Attacker creates token with transfer_fee_basis_points = 8000 (80% fee)
- Attacker markets the token as a legitimate project
- Users buy 1000 tokens for 1 SOL through the bonding curve
- Due to transfer fees, users receive only 200 tokens (80% lost to fees)
- The attacker withdraws the accumulated fees from the withheld authority
- Users are left with worthless tokens and no recourse

Recommendation

Add validation in setting the fee parameters at the time of bonding curve creation, and assert a maximum limit on the fee a bonding curve creator can receive

Status

Resolved

[M-03] Fee Charging Inconsistency

Description

The bonding curve implements inconsistent fee charging between buy and sell operations. Buy instructions charge fees as an additional payment on top of the input amount, while sell instructions deduct fees from the output amount. This creates economic asymmetry and results in double fee collection.

Vulnerability Details

In `buy.rs` and `buy_lock.rs`

```
// First transfer: User pays input amount
invoke(&transfer(
    &ctx.accounts.payer.key(),
    &ctx.accounts.bonding_curve.key(),
    lamports_in_amt,  // ← Full input amount
), ...)?;

// Second transfer: User pays fee separately
let fee_lamports = lamports_in_amt as f64 * swap_fee as f64;
invoke(&transfer(
    &ctx.accounts.payer.key(),
    &ctx.accounts.fee_pool.key(),
    fee_lamports as u64,  // ← Additional fee payment
), ...)?;
```

Sell Instruction:

```
// Calculate fee from output amount
let fee_lamports = (estimated_out_lamports as f64 * swap_fee as f64) as u64;
```

```
// Deduct fee from output
ctx.accounts.bonding_curve.sub_lamports(estimated_out_lamports)?;
ctx.accounts.fee_pool.add_lamports(fee_lamports)?;
ctx.accounts.payer.add_lamports(estimated_out_lamports - fee_lamports)?;    // ← Fee deducted
```

Impact

Buy: User pays 1.01 SOL → gets tokens worth 1 SOL
Sell: User gets 0.99 SOL ← gives tokens worth 1 SOL
Round trip loss: 0.02 SOL (2%)

- Asymmetric user experience: Buyers pay more than sellers receive for equivalent value
- Economic distortion: The bonding curve's price discovery is affected by the asymmetric fee structure
- Unfair trading costs: Users pay different effective fees depending on trade direction

Recommendation

As per the business logic, we recommend one of these two alternatives:

- Deduct fees from input in buy instructions (consistent with sell)
- Add fees to output in sell instruction (consistent with buy)

If the Fix is consistent with the fee in the Sell Instruction, then:

- Buy: User pays 1 SOL → gets tokens worth 0.99 SOL
- Sell: User gets 0.99 SOL ← gives tokens worth 1 SOL
- Round trip loss: 0.01 SOL (1%)

Status

Resolved

Low

[L-01] update_wallets.rs - Wrong event emission in `init_update_ponz_token_mint_authority_wallet`

Description

In `init_update_ponz_token_mint_authority_wallet`, the debug log uses:

```
msg!(
    "pending_ponz_token_mint_authoritY_wallet: {:?}",
    ctx.accounts
        .global_configuration
        .ponz_token_mint_authority_wallet
);
```

which prints the current `ponz_token_mint_authority_wallet` instead of the newly set `pending_ponz_token_mint_authority_wallet`.

In contrast, the other `init_update_*` handlers correctly log their `.pending_*` fields, leading to inconsistent and misleading log output.

Recommendation

Update the msg! call to reference and display `pending_ponz_token_mint_authority_wallet` so that logs accurately reflect the value being set.

Status

Resolved

[L-02] state/bonding_curve.rs - Arithmetic Overflow in Price Calculation could cause Panic

Description

The `calculate_price()` function performs arithmetic operations that could overflow `u64` before being cast to `u128`, or underflow `u128` during subtraction. With overflow checks enabled in `Cargo.toml`, these operations will panic the entire transaction instead of handling errors gracefully.

```
pub fn calculate_price(&self) -> f64 {

    let up = self.init_virtual_sol as u128 + self.sol_reserves as u128;

    let up_f64 = up.div(LAMPORTS_PER_SOL as u128) as f64;



//  Can panic if the value underflows

    let down = self.init_virtual_token as u128

        - (self.token_supply.mul(99).div(100) as u128 - self.token_reserves as u128);

  // If down_f64 becomes 0, division panics

      up_f64 / down_f64  // ← Panic if down_f64 == 0
```

Recommendation

Add overflow checks and graceful error handling rather than relying on default overflow checks

Status

Resolved

Centralisation

The Ponz project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Ponz project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au